

RELATÓRIO DO PROJETO FEItv

Disciplina: CCP110 – Fundamentos de Algoritmos

Bruno Vieira – RA: 52.126.025-7

Projeto FEItv

https://drive.google.com/file/d/1qYW9TDpV0_x1YWf3DnZXUcEdk72SdbGT/view?usp=sharing

1. Introdução

O projeto FEItv foi desenvolvido com o objetivo de construir uma plataforma simples de gerenciamento de informações sobre vídeos, inspirada em serviços de streaming como Netflix e YouTube. A proposta principal do sistema não é reproduzir vídeos, mas sim permitir o compartilhamento e organização de informações relacionadas a filmes e séries.

O sistema foi desenvolvido integralmente em Python utilizando arquivos de texto (.txt) para armazenamento permanente de dados, conforme solicitado nas especificações do projeto.

O projeto foi realizado individualmente, portanto foram implementadas apenas as funcionalidades destinadas aos usuários comuns, sem incluir as funcionalidades de administrador.

2. Objetivos do Projeto

Os principais objetivos do sistema FEItv são:

- Permitir cadastro e login de usuários;
 - Permitir busca de vídeos por diferentes critérios;
 - Permitir curtidas e remoção de curtidas;
 - Permitir gerenciamento de favoritos;
 - Persistir todas as informações em arquivos de texto;
 - Trabalhar conceitos fundamentais de programação em Python;
 - Aplicar estruturas condicionais, laços de repetição, funções e manipulação de arquivos.
-

3. Tecnologias Utilizadas

3.1 Linguagem Python

Toda a lógica do sistema foi desenvolvida em Python devido à simplicidade da linguagem e facilidade de manipulação de arquivos.

3.2 Arquivos TXT

Os dados do sistema são armazenados em arquivos de texto, garantindo persistência mesmo após o encerramento do programa.

Arquivos utilizados:

- usuarios.txt
- videos.txt
- curtidas.txt
- favoritos.txt

4. Estrutura do Projeto

A estrutura de diretórios foi organizada para facilitar a separação das funcionalidades.

```
FEItv/  
├── main.py  
├── usuarios.txt  
├── videos.txt  
├── curtidas.txt  
├── favoritos.txt  
└── funcoes/  
    ├── usuarios.py  
    ├── videos.py  
    ├── curtidas.py  
    └── favoritos.py
```

Descrição dos arquivos:

- main.py: arquivo principal responsável pelo menu do sistema;
- usuarios.py: funções relacionadas ao cadastro e login;
- videos.py: funções de listagem e busca de vídeos;
- curtidas.py: gerenciamento de curtidas;
- favoritos.py: gerenciamento de favoritos;
- arquivos .txt: armazenamento permanente dos dados.

5. Funcionamento Geral do Sistema

O programa funciona por meio de um menu interativo no terminal.

Ao iniciar o sistema, o usuário visualiza as opções disponíveis:

```
print("1 - Cadastrar usuário")
print("2 - Login")
print("3 - Listar vídeos")
print("4 - Buscar vídeo")
print("5 - Curtir vídeo")
print("6 - Favoritos")
print("0 - Sair")
```

O sistema permanece em execução utilizando um laço de repetição `while True` até que o usuário escolha sair.

6. Desenvolvimento das Funcionalidades

6.1 Cadastro de Usuário

A funcionalidade de cadastro permite que novos usuários sejam registrados no sistema.

Dados solicitados:

- Nome
- Email
- Senha

Os dados são armazenados no arquivo `usuarios.txt`.

Exemplo:

```
Bruno;bruno@gmail.com;123
Maria;maria@gmail.com;456
```

Trecho principal da função:

```
with open("usuarios.txt", "a") as arquivo:
    arquivo.write(f"{nome};{email};{senha}\n")
```

Explicação:

- `open()`: abre o arquivo;
 - `"a"`: modo append, utilizado para adicionar informações;
 - `write()`: grava os dados no arquivo;
 - `\n`: quebra de linha para separar os usuários.
-

6.2 Login de Usuário

A funcionalidade de login verifica se o email e senha digitados existem no arquivo usuarios.txt.

O sistema percorre todas as linhas utilizando um laço for.

Trecho principal:

```
for usuario in usuarios:
    dados = usuario.strip().split(";")
```

Explicação:

- strip(): remove espaços e quebras de linha;
- split(";"): separa os dados utilizando o ponto e vírgula.

Se o login estiver correto:

```
print("Login realizado!")
```

Caso contrário:

```
print("Email ou senha incorretos.")
```

6.3 Listagem de Vídeos

O sistema possui vídeos previamente cadastrados no arquivo videos.txt.

Exemplo:

```
1;Breaking Bad;Série;Drama
2;Interestelar;Filme;Ficção
3;The Batman;Filme;Ação
```

Cada linha possui:

- ID do vídeo
- Nome
- Tipo
- Gênero

A função listar_videos() percorre o arquivo e exibe os dados formatados.

Trecho utilizado:

```
for video in videos:
```

O laço percorre todas as linhas do arquivo.

6.4 Busca de Vídeos

A funcionalidade de busca foi ampliada para permitir pesquisas por:

- Nome
- ID
- Tipo
- Gênero

O usuário escolhe a opção desejada através de um submenu.

Exemplo:

```
print("1 - Buscar por nome")
print("2 - Buscar por ID")
print("3 - Buscar por tipo")
print("4 - Buscar por gênero")
```

O sistema realiza comparações utilizando estruturas condicionais.

Exemplo:

```
if busca == dados[0]:
```

Caso o vídeo seja encontrado, as informações são exibidas.

Caso contrário:

```
print("Vídeo não encontrado.")
```

6.5 Curtidas

A funcionalidade de curtidas permite que usuários curtam ou removam curtidas de vídeos.

Antes de realizar a curtida, o sistema verifica se o vídeo existe no arquivo videos.txt.

Validação:

```
if not video_encontrado:
    print("Vídeo não encontrado.")
```

As curtidas são armazenadas em curtidas.txt.

Exemplo:

```
bruno@gmail.com; 3
```

O sistema também verifica se o usuário já curtiu o vídeo.

Se já existir:

```
print("Curtida removida.")
```

Caso contrário:

```
print("Vídeo curtido!")
```

6.6 Sistema de Favoritos

O sistema de favoritos permite:

- Adicionar vídeos;
- Remover vídeos;
- Visualizar favoritos.

As informações são armazenadas em favoritos.txt.

Exemplo:

```
bruno@gmail.com; 2
```

O sistema verifica se o vídeo existe antes de adicioná-lo aos favoritos.

Validação:

```
if not video_encontrado:  
    print("Vídeo não encontrado.")
```

Também foi implementada validação para listas vazias.

Caso o usuário não tenha favoritos:

```
print("Você não adicionou nenhum vídeo aos favoritos.")
```

Ao remover favoritos, o sistema verifica:

- Se o vídeo existe;
- Se o vídeo realmente está nos favoritos.

Mensagem utilizada:

```
print("Esse vídeo não está nos favoritos.")
```

7. Manipulação de Arquivos

Um dos principais objetivos do projeto foi trabalhar persistência de dados.

Todos os dados permanecem salvos mesmo após fechar o programa.

Principais modos utilizados:

- "r" → leitura;
- "a" → adicionar informações;
- "w" → sobrescrever informações.

Exemplo:

```
with open("favoritos.txt", "w") as arquivo:
```

O uso do with garante fechamento automático do arquivo.

8. Estruturas de Programação Utilizadas

Durante o desenvolvimento foram utilizados diversos conceitos fundamentais de programação.

8.1 Estruturas Condicionais

Exemplo:

```
if opcao == "1":
```

Utilizadas para controlar decisões no sistema.

8.2 Laços de Repetição

Exemplo:

```
while True:
```

Utilizado para manter o menu em execução.

Outro exemplo:

```
for video in videos:
```

Utilizado para percorrer listas e arquivos.

8.3 Funções

O sistema foi dividido em funções para melhorar organização e reutilização de código.

Exemplo:

```
def cadastrar_usuario():
```

8.4 Manipulação de Strings

Funções utilizadas:

- lower()
- split()
- strip()

Essas funções ajudaram na busca e tratamento dos dados.

9. Conclusão

O desenvolvimento do projeto FEItv permitiu aplicar diversos conceitos fundamentais da programação utilizando Python.

Durante o projeto foi possível trabalhar:

- Estruturas condicionais;
- Estruturas de repetição;
- Funções;
- Manipulação de arquivos;
- Organização modular;
- Validação de dados.

Além disso, o projeto ajudou no desenvolvimento do raciocínio lógico e na compreensão prática de como sistemas simples funcionam internamente.

O resultado final foi um sistema funcional, organizado e compatível com todos os requisitos do projeto individual.

10. Referências

- Documentação oficial da linguagem Python
- Material das aulas da disciplina CCP110
- Conteúdo sobre manipulação de arquivos em Python

- Exercícios desenvolvidos durante o semestre